

Práctica:

Mi Primer Agente de IA

Construcción de un Agente de Soporte con Python y Gemini

Objetivo

Construir un agente de inteligencia artificial que lea automáticamente las quejas o sugerencias de un cliente registradas en un Google Form, las analice usando la API de Gemini (para clasificarlas y detectar el sentimiento), y registre ese análisis en una base de datos (Google Sheets). Todo el proceso se implementará con código Python que el alumno escribirá en su propia computadora.

Funcionamiento del Agente



Herramientas y Cuentas Requeridas

Herramienta	Para qué sirve	Costo
Cuenta de Google (Gmail)	Google Forms, Google Sheets y Google Cloud Console	Gratuito
Google Cloud Console	Crear las credenciales OAuth de acceso a Google Sheets	Gratuito
Google AI Studio	Obtener la API Key de Gemini (el 'cerebro' del agente)	Gratuito
Python 3.9 o superior	Ejecutar el script del agente en la computadora	Gratuito

Instalación de Dependencias Python

Antes de comenzar, abrir una terminal (CMD o PowerShell en Windows) y ejecutar el siguiente comando para instalar todas las librerías necesarias:

```
pip install gspread google-auth google-genai schedule
```

Librería	Función en el agente
gspread	Leer y escribir en Google Sheets
google-auth	Autenticar el script con las credenciales de Google
google-genai	Conectar con la API de Gemini para el análisis de IA
schedule	Ejecutar el agente automáticamente cada cierto tiempo

FASE 1: Preparar la Entrada y la Salida (Google Forms y Sheets)

Paso 1.1 — Crear el Formulario de Quejas (Google Forms)

- Ir a **forms.google.com** e iniciar sesión con la cuenta de Google.
- Crear un formulario nuevo. Título: "**Formulario de Quejas y Sugerencias**".
- Añadir una pregunta de tipo **Respuesta corta** con el texto: *Tu Nombre* (campo opcional).
- Añadir una pregunta de tipo **Párrafo** con el texto: *Describe tu queja, sugerencia o comentario*. Esta es la entrada clave que el agente analizará.

Paso 1.2 — Crear la Base de Datos (Google Sheets)

- Dentro del formulario, ir a la pestaña **Respuestas**.
- Hacer clic en el ícono verde de Google Sheets y seleccionar "**Crear una nueva hoja de cálculo**".
- Nombrarla exactamente: "**Respuestas de Quejas (Base de Datos)**".
- La hoja tendrá las columnas A, B y C generadas automáticamente por el Form. Añadir manualmente las siguientes tres columnas:

Columna	Encabezado a escribir	¿Quién la llena?
A	Marca temporal	Google Forms (automático)
B	Tu Nombre	Google Forms (automático)
C	Describe tu queja...	Google Forms (automático)
D	Clasificación IA	El agente de Python
E	Sentimiento IA	El agente de Python
F	Agente Ejecutado	El agente de Python (escribe 'SI')

Paso 1.3 — Enviar un Dato de Prueba

- En el formulario, hacer clic en el ícono del **ojo** (Vista previa).
- Llenar el formulario con los siguientes datos de prueba:

Nombre: Cliente de Prueba
Comentario: Mi pedido llegó tarde y la caja estaba rota. Muy mal servicio.

- Verificar que la respuesta aparezca en la hoja de cálculo en las columnas A, B y C.

FASE 2: Configurar el Acceso a Google Sheets (OAuth 2.0)

OAuth 2.0 permite que el script acceda a Google Sheets autenticándose como tú, el usuario. La primera vez que se ejecute el agente, se abrirá el navegador para confirmar el acceso. Después de eso, el token se guarda y no vuelve a preguntar.

Paso 2.1 — Entrar a Google Cloud Console

- Ir a **console.cloud.google.com** e iniciar sesión con la cuenta de Google normal (Gmail). No se requiere una cuenta nueva.
- Si es la primera vez, aceptar los términos de servicio.

Nota: Google puede mostrar una pantalla para activar su 'prueba gratuita de \$300'. Esta pantalla se puede ignorar o cerrar. Las APIs que se usan en esta práctica son completamente gratuitas y no requieren método de pago.

Paso 2.2 — Crear un Proyecto

- En la barra superior, hacer clic en el **selector de proyectos** (junto al logo de Google Cloud).
- Hacer clic en **"Proyecto nuevo"** (arriba a la derecha del modal).
- Asignarle el nombre: **Agente de Soporte** y hacer clic en **Crear**.
- Asegurarse de que el nuevo proyecto esté seleccionado en la barra superior antes de continuar.

Paso 2.3 — Habilitar las APIs Necesarias

- En el menú de la izquierda, ir a **"APIs y servicios"** → **"Biblioteca"**.
- Buscar **Google Sheets API**, hacer clic en ella y luego en **"Habilitar"**.
- Regresar a la Biblioteca, buscar **Google Drive API** y también **"Habilitar"**.

Paso 2.4 — Configurar la Pantalla de Consentimiento OAuth

- Ir a **"APIs y servicios"** → **"OAuth consent screen"** (Pantalla de consentimiento de OAuth).
- En la sección **"Audience"** (o tipo de usuario), seleccionar **"External"** y hacer clic en **"Crear"**.
- En la siguiente pantalla, llenar únicamente los campos obligatorios: **nombre de la app** (cualquier nombre, ej. AgenteSoporte) y **correo electrónico**.
- Hacer clic en **"Guardar y continuar"** en todos los pasos hasta llegar al resumen final.

Paso 2.5 — Crear las Credenciales OAuth

- Ir a **"APIs y servicios"** → **"Credenciales"**.
- Hacer clic en **"+ Crear credenciales"** → seleccionar **"ID de cliente de OAuth 2.0"**.
- En **Tipo de aplicación**, seleccionar **"Aplicación de escritorio"** (Desktop app).
- Asignar cualquier nombre (ej. *agente-soporte*) y hacer clic en **"Crear"**.
- En la ventana que aparece, hacer clic en **"Descargar JSON"**.
- **Renombrar el archivo descargado a client_secrets.json** y colocarlo en la misma carpeta que el script.

Nota: El archivo client_secrets.json contiene las credenciales de la aplicación. No compartirlo ni subirlo a internet (GitHub, Google Drive público, etc.).

Paso 2.6 — Agregarse como Usuario de Prueba

Como la aplicación está en modo de prueba, es necesario agregar tu propio correo como usuario autorizado para evitar un error de acceso al ejecutar el script.

- Ir a **"APIs y servicios"** → **"OAuth consent screen"**.
- Buscar la sección **"Audience"** y dentro de ella **"Test users"**.
- Hacer clic en **"+ Add Users"** y escribir tu propio correo de Gmail.

- Hacer clic en "**Guardar**".

Nota: La primera vez que se ejecute el script, se abrirá el navegador para iniciar sesión con tu cuenta de Google y autorizar el acceso. Esto solo ocurre una vez.

FASE 3: Obtener la API Key de Gemini

La API Key de Gemini es la llave que permite al script comunicarse con la inteligencia artificial de Google. Su obtención es gratuita.

- Ir a aistudio.google.com/app/api-keys e iniciar sesión con la cuenta de Google.
- Hacer clic en "**Crear clave de API**".
- Seleccionar el proyecto **Agente de Soporte** (creado en la Fase 2).
- Copiar la clave generada. Tendrá un formato similar a:

```
AIzaSyAbc123xyzEjemploDeClaveReal456
```

Nota: Guardar la clave en un lugar seguro. Se necesitará en la Fase 4. No compartirla públicamente, ya que representa el acceso a la cuenta de IA.

FASE 4: Escribir el Script del Agente

En esta fase se construye el script **agente_soporte.py**. A continuación se describe lo que debe hacer cada sección del programa. Se recomienda usar Visual Studio Code como editor.

Paso 4.1 — Organización de Archivos

Crear una carpeta para el proyecto y asegurarse de que contenga los siguientes archivos:

```
mi_agente_soporte/  
  agente_soporte.py      <-- el script que se escribirá en esta fase  
  client_secrets.json    <-- descargado en la Fase 2
```

Paso 4.2 — Sección de Importaciones

Al inicio del archivo se deben importar todas las librerías necesarias para que el script funcione. Las librerías requeridas son:

- **json** — para convertir la respuesta de Gemini en un diccionario Python.
- **os** — para verificar que el archivo de credenciales existe antes de ejecutar.
- **time** — para hacer pausas entre ciclos del agente.
- **schedule** — para programar la ejecución automática cada cierto tiempo.
- **gsread** — para leer y escribir en Google Sheets.
- **google.genai** y **google.genai.types** — para conectarse con la API de Gemini.

Paso 4.3 — Sección de Configuración

Definir las siguientes constantes al inicio del script. **El alumno solo debe modificar el valor de la API Key**, reemplazando el texto de marcador por la clave obtenida en la Fase 3:

- **ARCHIVO_CREDENCIALES** — nombre del archivo OAuth descargado en la Fase 2 (*client_secrets.json*).
- **NOMBRE_HOJA** — nombre exacto de la hoja de cálculo creada en la Fase 1 (con mayúsculas y espacios exactos).
- **GEMINI_API_KEY** — la API Key de Gemini obtenida en la Fase 3. **Este es el único valor que se debe reemplazar.**
- **INTERVALO_MINUTOS** — cada cuántos minutos el agente revisará si hay nuevas quejas (valor sugerido: 15).
- **Índices de columnas** — constantes numéricas que indican en qué columna del Sheet escribe el agente: Clasificación IA en la columna D (4), Sentimiento IA en la E (5) y Agente Ejecutado en la F (6).

Paso 4.4 — Función: Conectar con Google Sheets

Escribir una función llamada **conectar_sheets()** que realice lo siguiente:

- Autenticar el script con Google usando **OAuth 2.0** a través de gspread, pasando como parámetro el nombre del archivo de credenciales.
- Abrir la hoja de cálculo usando el nombre definido en **NOMBRE_HOJA**.
- Retornar el objeto de la primera hoja (sheet1) para que el resto del script pueda leer y escribir en ella.

Nota: La primera vez que se ejecute el script, gspread abrirá automáticamente el navegador para que el alumno inicie sesión con su cuenta de Google y autorice el acceso.

Paso 4.5 — Función: Analizar con Gemini

Escribir una función llamada **analizar_con_gemini(comentario)** que realice lo siguiente:

- **Instrucción del sistema:** indicarle a Gemini que es un analista de soporte al cliente y que su única tarea es devolver una respuesta en formato JSON, sin texto adicional ni explicaciones.
- **Instrucción del usuario:** enviar el comentario recibido y pedirle a Gemini que lo clasifique en una de tres categorías (*Ventas*, *Soporte Técnico* o *Logística*) y que detecte el sentimiento (*Positivo*, *Negativo* o *Neutro*).
- **Llamada a la API:** usar el cliente de Gemini para enviar ambas instrucciones al modelo *gemini-2.5-flash*, usando *GenerateContentConfig* para incluir la instrucción del sistema.
- **Limpiar la respuesta:** quitar posibles bloques de código (`` ` `` o ````json`) que Gemini pueda agregar alrededor del JSON.
- **Retornar** un diccionario Python con las claves *"clasificacion"* y *"sentimiento"*, convirtiendo el texto JSON con *json.loads()*.

Paso 4.6 — Función: Agente Principal

Escribir una función llamada **ejecutar_agente()** que orqueste el ciclo completo del agente. Esta función debe realizar los siguientes pasos en orden:

- **Conectar:** llamar a *conectar_sheets()* para obtener la hoja. Si ocurre un error de conexión, mostrarlo en pantalla y terminar el ciclo.
- **Leer:** obtener todas las filas de la hoja con *get_all_values()* y omitir la primera fila (que contiene los encabezados).
- **Filtrar:** recorrer cada fila y revisar si la columna F está vacía. Si ya contiene "SI", significa que esa fila fue procesada antes: omitirla y continuar con la siguiente.

- **Analizar:** para cada fila nueva, extraer el comentario de la columna C y enviarlo a `analizar_con_gemini()`. Mostrar en pantalla el nombre del cliente y los resultados obtenidos.
- **Escribir:** guardar la clasificación en la columna D, el sentimiento en la columna E, y escribir "SI" en la columna F para marcar la fila como procesada.
- **Manejo de errores:** si Gemini no devuelve un JSON válido en alguna fila, mostrar el error en pantalla y continuar con la siguiente fila sin detener el agente.

Paso 4.7 — Punto de Entrada del Script

Al final del archivo, agregar el bloque principal (`if __name__ == "__main__"`) que realice lo siguiente al ejecutar el script:

- **Validar la API Key:** si el valor de `GEMINI_API_KEY` aún es el texto de marcador, mostrar un mensaje de error y detener el programa.
- **Validar el archivo de credenciales:** verificar que `client_secrets.json` existe en la carpeta. Si no se encuentra, mostrar un mensaje de error y detener el programa.
- **Inicializar el cliente de Gemini:** crear el cliente usando la API Key, como variable global para que las funciones puedan usarlo.
- **Ejecutar inmediatamente:** llamar a `ejecutar_agente()` una primera vez al arrancar, sin esperar el primer intervalo.
- **Programar ejecuciones automáticas:** usar `schedule` para que el agente se ejecute automáticamente cada `INTERVALO_MINUTOS` minutos.
- **Mantener el script corriendo:** entrar en un bucle infinito que cada 30 segundos revisa si hay tareas programadas pendientes por ejecutar.

FASE 5: Ejecutar y Probar el Agente

Paso 5.1 — Ejecutar el Script

Abrir una terminal (CMD o PowerShell), navegar a la carpeta del proyecto y ejecutar el script con el siguiente comando:

```
python agente_soporte.py
```

Si el script está correctamente escrito, en la terminal se debe ver un mensaje de inicio indicando que el agente está corriendo y cada cuántos minutos revisará la hoja. Luego mostrará el resultado del análisis de cada fila nueva encontrada: el nombre del cliente, la clasificación y el sentimiento detectados, y una confirmación de que los resultados fueron escritos en Google Sheets.

Paso 5.2 — Enviar Nuevas Quejas de Prueba

Abrir el Google Form en la vista previa y enviar al menos tres respuestas de prueba con distintos tipos de comentarios:

Nombre	Comentario de prueba	Resultado esperado
Ana	No entiendo cómo funciona la nueva app, el botón de ayuda no sirve.	Soporte Técnico / Negativo
Carlos	Me atendió un vendedor llamado Juan y fue increíble, ¡resolvió todo! 10/10.	Ventas / Positivo

María	Mi paquete lleva 2 semanas sin llegar y nadie me da información.	Logística / Negativo
-------	--	----------------------

Paso 5.3 — Verificar los Resultados en Google Sheets

- Detener el script con **Ctrl+C** y volver a ejecutarlo para procesar las nuevas respuestas.
- Abrir la hoja de cálculo y verificar que las columnas D, E y F se llenaron correctamente con el análisis de la IA.

Solución de Errores Comunes

Error	Causa probable	Solución
FileNotFoundError: client_secrets.json	El archivo no está en la misma carpeta que el script	Mover client_secrets.json junto a agente_soporte.py
Error 403: access_denied al abrir el navegador	El correo no fue agregado como usuario de prueba en OAuth	Repetir el Paso 2.6 y agregar el correo en Test users
SpreadsheetNotFound	El nombre de la hoja no coincide exactamente con NOMBRE_HOJA	Verificar mayúsculas, tildes y espacios en NOMBRE_HOJA
API key not valid (Gemini)	La API Key es incorrecta o tiene espacios adicionales	Copiar la clave de nuevo desde AI Studio sin espacios
JSONDecodeError	Gemini no devolvió JSON puro en esa respuesta	Volver a ejecutar el agente. El error es poco frecuente
ModuleNotFoundError	No se instalaron las dependencias de Python	Ejecutar: pip install gspread google-auth google-genai schedule

Comparación: Make.com vs Script Python

Esta práctica replica exactamente el mismo agente de la guía original con Make.com. La siguiente tabla muestra la equivalencia entre cada componente:

Componente en Make.com	Equivalente en Python
Módulo Google Forms – Watch new responses	hoja.get_all_values() + filtro de columna F vacía
Módulo Google AI Studio – Generate Content	cliente_gemini.models.generate_content(model=..., contents=..., config=...)
Módulo JSON – Parse JSON	json.loads(respuesta.text)
Módulo Google Sheets – Add a Row	hoja.update_cell(fila, columna, valor)
Scheduling – Cada 15 minutos	schedule.every(15).minutes.do(ejecutar_agente)
Conectar cuenta de Google	Archivo client_secrets.json (OAuth 2.0 — abre el navegador la primera vez)